Worksheet 0

Shiv Kumar Yadav
Swoyam Pokharel

Date created: 2026-03-02
Date modified: 2026-03-02

Contents

1. Exercise on Functions . . . 2
   1.1. Task 1: Unit Converter . . . 2
   1.2. Task 2: List Calculator . . . 3
2. Exercise on List Manipulation . . . 4
   2.1. Slicing Utils . . . 4
3. Exercise on Nested Lists . . . 5
4. NumPy . . . 7
   4.1. Problem 1: Array Creation . . . 7
   4.2. Problem 2: Array Manipulation . . . 8
   4.3. Problem 3: Array Operations . . . 9
   4.4. Problem 4: Matrix Operations . . . 11
5. Experiment: How Fast is NumPy? . . . 13
   5.1. Setup . . . 13
   5.2. 1. Element-wise Addition . . . 13
   5.3. 2. Element-wise Multiplication . . . 13
   5.4. 3. Dot Product . . . 14
   5.5. 4. Matrix Multiplication . . . 14

1. Exercise on Functions

1.1. Task 1: Unit Converter

Convert between length (m ↔ ft), weight (kg ↔ lbs), and volume (L ↔ gal). Includes error handling for invalid input.

```python
def convert_measurements():
    """
        Displays a menu of conversion options and prompts the user to select one.
        Supports conversions between metric and imperial units:
        - Length: meters <-> feet
        - Weight: kilograms <-> pounds
        - Volume: liters <-> gallons

        Raises:
                ValueError: If the user enters an invalid choice or non-numeric input.
        """
    conversions = {
        "m -> ft":  lambda n: n * 3.280,
        "ft -> m":  lambda n: n * 0.3048,
        "kg -> lbs": lambda n: n * 2.20462,
        "lbs -> kg": lambda n: n * 0.453592,
        "l -> gal": lambda n: n * 0.264172,
        "gal -> l": lambda n: n * 3.78541
    }

    for i, k in enumerate(conversions):
        print(f"{i}) {k}")

    try:
        operation = int(input("Enter your choice: "))
        num = float(input("Enter a number: "))

        if operation < 0 or operation >= len(conversions):
            raise ValueError("Invalid choice. Please select a valid option.")

        key = list(conversions.keys())[operation]
        result = conversions[key](num)
        print(f"{num} {key} = {result}")

    except ValueError as ve:
        print(f"Input Error: {ve}")
    except Exception as e:
        print(f"Error: {e}")

convert_measurements()
```

```
--- Output ---
0) m -> ft
1) ft -> m
2) kg -> lbs
3) lbs -> kg
4) l -> gal
5) gal -> l
Enter your choice: 3
Enter a number: 23423423
23423423.0 lbs -> kg = 10624677.285416
```

1.2. Task 2: List Calculator

Perform sum, average, maximum, or minimum on a user-supplied list of numbers.

```python
def calculator():
    """
        Prompts the user to select an operation (sum, average, maximum, or minimum)
        and enter a list of numbers. Performs the selected operation and displays
        the result.

        Operations:
                1) Sum     - Returns the sum of all numbers
                2) Average - Returns the arithmetic mean
                3) Maximum - Returns the largest number
                4) Minimum - Returns the smallest number

        Raises:
                ValueError: If the user enters an invalid choice, non-numeric input,
                                        or an empty list is provided.
        """
    operations = {
        "1": ("Sum",     sum),
        "2": ("Average", lambda n: sum(n) / len(n)),
        "3": ("Maximum", max),
        "4": ("Minimum", min)
    }

    print("1) Sum\n2) Average\n3) Maximum\n4) Minimum")
    op = input("Choose operation: ")

    if op not in operations:
        print("Invalid choice.")
        return

    try:
        nums = list(map(float, input("Enter numbers separated by spaces: ").split()))
        if not nums:
```

```python
            raise ValueError("Empty list")
        print(f"Result: {operations[op][1](nums)}")
    except ValueError as e:
        print(f"Error: {e}")

calculator()
```

```
--- Output ---
1) Sum
2) Average
3) Maximum
4) Minimum
Choose operation: 2
Enter numbers separated by spaces: 3 1 2 3 5
Result: 2.8
```

---

2. Exercise on List Manipulation

2.1. Slicing Utils

```python
def extract_every_other(lst):  return lst[::2]
def get_sublist(lst, start, end): return lst[start:end+1]
def reverse_list(lst):         return lst[::-1]
def remove_first_last(lst):    return lst[1:-1]
def get_first_n(lst, n):       return lst[:n]
def get_last_n(lst, n):        return lst[-n:]
def reverse_skip(lst):         return lst[-2::-2]

print(extract_every_other([1, 2, 3, 4, 5, 6]))
print(get_sublist([1, 2, 3, 4, 5, 6], 2, 4))
print(reverse_list([1, 2, 3, 4, 5]))
print(remove_first_last([1, 2, 3, 4, 5]))
print(get_first_n([1, 2, 3, 4, 5], 3))
print(get_last_n([1, 2, 3, 4, 5], 2))
print(reverse_skip([1, 2, 3, 4, 5, 6]))
```

```
-- Output--
[1, 3, 5]
[3, 4, 5]
[5, 4, 3, 2, 1]
[2, 3, 4]
[1, 2, 3]
[4, 5]
[5, 3, 1]
```

---

3. Exercise on Nested Lists

```python
def flatten(lst):
    return [x for sub in lst for x in sub]

def access_nested_element(lst, indices):
    result = lst
    for index in indices:
        result = result[index]
    return result

def sum_nested(lst):
    total = 0
    for item in lst:
        if isinstance(item, list):
            total += sum_nested(item)
        else:
            total += item
    return total

def remove_element(lst, elem):
    result = []
    for item in lst:
        if isinstance(item, list):
            result.append(remove_element(item, elem))
        elif item != elem:
            result.append(item)
    return result

def find_max(lst):
    max_val = float('-inf')
    for item in lst:
        if isinstance(item, list):
            max_val = max(max_val, find_max(item))
        else:
            max_val = max(max_val, item)
    return max_val

def count_occurrences(lst, elem):
    count = 0
    for item in lst:
        if isinstance(item, list):
            count += count_occurrences(item, elem)
        elif item == elem:
            count += 1
    return count

def deep_flatten(lst):
    result = []
    for item in lst:
        if isinstance(item, list):
            result.extend(deep_flatten(item))
        else:
```

```python
            result.append(item)
    return result

def average_nested(lst):
    flat = deep_flatten(lst)
    return sum(flat) / len(flat)


print(flatten([[1, 2], [3, 4], [5]]))
print(access_nested_element([[1,2,3],[4,5,6],[7,8,9]], [1, 2]))
print(sum_nested([[1, 2], [3, [4, 5]], 6]))
print(remove_element([[1, 2], [3, 2], [4, 5]], 2))
print(find_max([[1, 2], [3, [4, 5]], 6]))
print(count_occurrences([[1, 2], [2, 3], [2, 4]], 2))
print(deep_flatten([[[1, 2], [3, 4]], [[5, 6], [7, 8]]]))
print(average_nested([[1, 2], [3, 4], [5, 6]]))
```

```
--- Output ---
[1, 2, 3, 4, 5]
6
21
[[1], [3], [4, 5]]
6
3
[1, 2, 3, 4, 5, 6, 7, 8]
3.5
```

4. NumPy

4.1. Problem 1: Array Creation

```python
import numpy as np

# 1. Empty 2x2 array
empty_arr = np.empty((2, 2))
print(empty_arr)
```

```
[[6.09668558e-317 0.00000000e+000]
 [1.28091851e+213 6.84479946e-310]]
```

```python
# 2. All-ones 4x2 array
ones_arr = np.ones((4, 2))
print(ones_arr)
```

```
[[1. 1.]
 [1. 1.]
 [1. 1.]
 [1. 1.]]
```

```python
# 3. Array filled with fill value
full_arr = np.full((3, 3), 7)
print(full_arr)
```

```
[[7 7 7]
 [7 7 7]
 [7 7 7]]
```

```python
# 4. Zeros like a given array
given = np.array([[1, 2], [3, 4]])
zeros_like_arr = np.zeros_like(given)
print(zeros_like_arr)
```

```
[[0 0]
 [0 0]]
```

```python
# 5. Ones like a given array
ones_like_arr = np.ones_like(given)
print(ones_like_arr)
```

```
[[1 1]
 [1 1]]
```

```python
# 6. Convert list to NumPy array
new_list = [1, 2, 3, 4]
numpy_arr = np.array(new_list)
print(numpy_arr)
```

```
[1 2 3 4]
```

4.2. Problem 2: Array Manipulation

```python
import numpy as np

# 1. Array with values 10-49
arr_10_49 = np.arange(10, 50)
print(arr_10_49)
```

```
[10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33
 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49]
```

```python
# 2. 3x3 matrix with values 0-8
matrix_0_8 = np.arange(9).reshape(3, 3)
print(matrix_0_8)
```

```
[[0 1 2]
 [3 4 5]
 [6 7 8]]
```

```python
# 3. 3x3 identity matrix
identity = np.eye(3)
print(identity)
```

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

```python
# 4. Random array of size 30, find mean
random_arr = np.random.random(30)
print(random_arr.mean())
```

```
0.438597
```

```python
# 5. 10x10 random array, min and max
random_10x10 = np.random.random((10, 10))
print(f"Min: {random_10x10.min():.6f}, Max: {random_10x10.max():.6f}")
```

```
Min: 0.005522, Max: 0.986887
```

```python
# 6. Zero array of size 10, replace 5th element with 1
zero_arr = np.zeros(10)
zero_arr[4] = 1
print(zero_arr)
```

```
[0. 0. 0. 0. 1. 0. 0. 0. 0. 0.]
```

```python
# 7. Reverse array
arr = np.array([1, 2, 0, 0, 4, 0])
reversed_arr = arr[::-1]
print(reversed_arr)
```

```
[0 4 0 0 2 1]
```

```python
# 8. 2-D array with 1 on border, 0 inside
border_arr = np.ones((5, 5))
border_arr[1:-1, 1:-1] = 0
print(border_arr)
```

```
[[1. 1. 1. 1. 1.]
 [1. 0. 0. 0. 1.]
 [1. 0. 0. 0. 1.]
 [1. 0. 0. 0. 1.]
 [1. 1. 1. 1. 1.]]
```

```python
# 9. 8x8 checkerboard pattern
checker = np.zeros((8, 8))
checker[::2, ::2]   = 1
checker[1::2, 1::2] = 1
print(checker)
```

```
[[1. 0. 1. 0. 1. 0. 1. 0.]
 [0. 1. 0. 1. 0. 1. 0. 1.]
 [1. 0. 1. 0. 1. 0. 1. 0.]
 [0. 1. 0. 1. 0. 1. 0. 1.]
 [1. 0. 1. 0. 1. 0. 1. 0.]
 [0. 1. 0. 1. 0. 1. 0. 1.]
 [1. 0. 1. 0. 1. 0. 1. 0.]
 [0. 1. 0. 1. 0. 1. 0. 1.]]
```

4.3. Problem 3: Array Operations

```python
import numpy as np

x = np.array([[1, 2], [3, 5]])
y = np.array([[5, 6], [7, 8]])
v = np.array([9, 10])
w = np.array([11, 12])
```

```python
# 1. Add
print(x + y)
```

```
[[ 6  8]
 [10 13]]
```

```python
# 2. Subtract
```

```python
print(x - y)
```

```
[[-4 -4]
 [-4 -3]]
```

```python
# 3. Multiply by scalar
print(x * 3)
```

```
[[ 3  6]
 [ 9 15]]
```

```python
# 4. Square each element
print(x ** 2)
```

```
[[ 1  4]
 [ 9 25]]
```

```python
# 5. Dot products
print(f"v . w = {np.dot(v, w)}")
print(f"x . v = {np.dot(x, v)}")
print(f"x . y =\n{np.dot(x, y)}")
```

```
v . w = 219
x . v = [29 77]
x . y =
[[19 22]
 [50 58]]
```

```python
# 6. Concatenate x and y along rows; stack v and w as columns
print(np.concatenate((x, y), axis=0))
print(np.column_stack((v, w)))
```

```
[[1 2]
 [3 5]
 [5 6]
 [7 8]]
[[ 9 11]
 [10 12]]
```

```python
# 7. Concatenate x and v
# x is 2-D (shape 2x2) while v is 1-D (shape 2).
# NumPy requires all arrays to share the same number of dimensions.
try:
    np.concatenate((x, v))
except Exception as e:
    print(f"Error: {e}")
```

```
Error: all the input arrays must have same number of dimensions,
but the array at index 0 has 2 dimension(s) and the array at
```

```
index 1 has 1 dimension(s)
```

4.4. Problem 4: Matrix Operations

```python
import numpy as np

A = np.array([[3, 4], [7, 8]])
B = np.array([[5, 3], [2, 1]])
```

```python
# 1. Prove A . A^-1 = I
A_inv = np.linalg.inv(A)
print(np.dot(A, A_inv))
```

```
[[1.00000000e+00 0.00000000e+00]
 [1.77635684e-15 1.00000000e+00]]
```

```python
# 2. Prove AB != BA
AB = np.dot(A, B)
BA = np.dot(B, A)
print(f"AB =\n{AB}\nBA =\n{BA}\nAB != BA: {not np.array_equal(AB, BA)}")
```

```
AB =
[[23 13]
 [51 29]]
BA =
[[36 44]
 [13 16]]
AB != BA: True
```

```python
# 3. Prove (AB)^T = B^T * A^T
AB_T  = AB.T
BT_AT = np.dot(B.T, A.T)
print(f"(AB)T =\n{AB_T}\nBT*AT =\n{BT_AT}\nEqual: {np.array_equal(AB_T,
BT_AT)}")
```

```
(AB)T =
[[23 51]
 [13 29]]
BT*AT =
[[23 51]
 [13 29]]
Equal: True
```

```python
# Solve system of linear equations using inverse method
# 2x - 3y +  z = -1
#  x -  y + 2z = -3
# 3x +  y -  z =  9

coeffs    = np.array([[2, -3, 1], [1, -1, 2], [3, 1, -1]])
```

```python
constants = np.array([-1, -3, 9])

solution_inv = np.dot(np.linalg.inv(coeffs), constants)
print(f"Solution via inverse (x, y, z): {solution_inv}")
```

```
Solution via inverse (x, y, z): [ 2.  1. -2.]
```

```python
# Using np.linalg.solve
solution_solve = np.linalg.solve(coeffs, constants)
print(f"Solution via linalg.solve (x, y, z): {solution_solve}")
```

```
Solution via linalg.solve (x, y, z): [ 2.  1. -2.]
```

```python
# Verification
x_s, y_s, z_s = solution_inv
print(f"2({x_s:.2f}) - 3({y_s:.2f}) + {z_s:.2f}  = {2*x_s - 3*y_s + z_s:.2f}
(expected -1)")
print(f"  {x_s:.2f} -   {y_s:.2f} + 2({z_s:.2f}) = {x_s - y_s + 2*z_s:.2f}
(expected -3)")
print(f"3({x_s:.2f}) +   {y_s:.2f} - {z_s:.2f}  = {3*x_s + y_s - z_s:.2f}
(expected  9)")
```

```
2(2.00) - 3(1.00) + -2.00  = -1.00  (expected -1)
  2.00 -   1.00 + 2(-2.00) = -3.00  (expected -3)
3(2.00) +   1.00 - -2.00   =  9.00  (expected  9)
```

---

5. Experiment: How Fast is NumPy?

5.1. Setup

```python
import time
import random
import numpy as np

SIZE = 1_000_000
MAT  = 1000

def timer(label, fn):
    start  = time.time()
    result = fn()
    end    = time.time()
    print(f"  {label:<30} {(end - start)*1000:.4f} ms")
    return result

py_a  = [random.random() for _ in range(SIZE)]
py_b  = [random.random() for _ in range(SIZE)]
np_a  = np.array(py_a)
np_b  = np.array(py_b)

py_m1 = [[random.random() for _ in range(MAT)] for _ in range(MAT)]
py_m2 = [[random.random() for _ in range(MAT)] for _ in range(MAT)]
np_m1 = np.array(py_m1)
np_m2 = np.array(py_m2)
```

5.2. 1. Element-wise Addition

```python
print("1. Element-wise Addition (size 1,000,000)")
timer("Python list", lambda: [a + b for a, b in zip(py_a, py_b)])
timer("NumPy array", lambda: np_a + np_b)
```

```
1. Element-wise Addition (size 1,000,000)
  Python list                    75.8698 ms
  NumPy array                    4.0989 ms
```

5.3. 2. Element-wise Multiplication

```python
print("2. Element-wise Multiplication (size 1,000,000)")
timer("Python list", lambda: [a * b for a, b in zip(py_a, py_b)])
timer("NumPy array", lambda: np_a * np_b)
```

```
2. Element-wise Multiplication (size 1,000,000)
  Python list                    59.7401 ms
  NumPy array                    2.5773 ms
```

5.4. 3. Dot Product

```python
print("3. Dot Product (size 1,000,000)")
timer("Python list", lambda: sum(a * b for a, b in zip(py_a, py_b)))
timer("NumPy array", lambda: np.dot(np_a, np_b))
```

```
3. Dot Product (size 1,000,000)
  Python list                    67.2050 ms
  NumPy array                    1.4241 ms
```

5.5. 4. Matrix Multiplication

```python
print("4. Matrix Multiplication (1000 x 1000)")
def py_matmul(A, B):
    n = len(A)
    C = [[0.0] * n for _ in range(n)]
    for i in range(n):
        for k in range(n):
            if A[i][k] == 0:
                continue
            for j in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

timer("Python list", lambda: py_matmul(py_m1, py_m2))
timer("NumPy array", lambda: np_m1 @ np_m2)
```

```
4. Matrix Multiplication (1000 x 1000)
  Python list                    108986.7711 ms
  NumPy array                    58.4090 ms
```

```
array([[248.41354013, 257.51759579, 252.73415443, ..., 251.22828762,
        248.19849501, 247.85047517],
       [244.02302257, 252.55309168, 248.44192676, ..., 245.90116679,
        246.53528278, 245.68427699],
       [249.45995735, 253.41159868, 251.59668811, ..., 238.9735616 ,
        241.9744867 , 242.6546252 ],
       ...,
       [247.43058301, 253.31716966, 248.57757713, ..., 241.92803161,
        244.18036446, 243.71404217],
       [241.79131323, 251.54820816, 251.14875884, ..., 241.41160045,
        241.19752429, 248.67845752],
       [254.81074302, 263.24777768, 256.69652542, ..., 253.77075173,
        254.0680978 , 251.20478225]])
```